



TKGR-RHETNE: A New Temporal Knowledge Graph Reasoning Model via Jointly Modeling Relevant Historical Event and Temporal Neighborhood Event Context

Jinze Sun¹, Yongpan Sheng^{1,2(✉)}, Ling Zhan¹, and Lirong He³

¹ College of Computer and Information Science, Southwest University, Chongqing 400715, China

ssunjinze@outlook.com, shengyp2011@gmail.com,
z10327@email.swu.edu.cn

² School of Big Data & Software Engineering, Chongqing University, Chongqing 401331, China

³ School of Information Science and Engineering, Chongqing Jiaotong University, Chongqing 400074, China
ronghe1217@gmail.com

Abstract. Temporal knowledge graph reasoning (TKGR) has been of great interest for its role in enriching the naturally incomplete temporal knowledge graph (TKG) by uncovering new events from existing ones with temporal information. At present, the majority of existing TKGR methods have attained commendable performance. Nevertheless, they still suffer from several problems, specifically their limited ability to adeptly capture intricate long-term event dependencies within the context of pertinent historical events, as well as to address the occurrence of an event with insufficient historical information or be influenced by other events. To alleviate such issues, we propose a novel TKGR method named TKGR-RHETNE, which jointly models the context of relevant historical events and temporal neighborhood events. In terms of the historical event view, we introduce an encoder based on the transformer Hawkes process and self-attention mechanism to effectively capture long-term event dependencies, thus modeling the event evolution process continuously. In terms of the neighborhood event view, we propose a neighborhood aggregator to model the potential influence between events with insufficient historical information and other events, which is implemented by integrating the random walk strategy with the TKG topological structure. Comprehensive experiments on five benchmark datasets demonstrate the superior performance of our proposed model (Code is publicly available at <https://github.com/wanwano/TKGR-RHETNE>).

1 Introduction

Knowledge graphs (KGs) provide significant real-world factual information using a multi-relational graph structure and have achieved remarkable promise in NLP or knowledge engineering-aware tasks. However, factual knowledge in real is continuously evolving, in the form of event knowledge, which has led to the development

and application of temporal knowledge graphs (TKGs). TKG offers fresh perspectives and valuable insights for many time-sensitive downstream applications, including time-sensitive semantic search [1], policymaking [4], and stock prediction [5].

With the advancement of the TKG, more and more researchers are starting to pay their attention to the TKG reasoning (TKGR) task to infer new event knowledge based on existing ones for filling in natural incomplete TKG and checking these knowledge consistencies. On the benefit of their representation learning ability, various learning-based methods associate temporal information with the facts, undoubtedly, this severely limits the ability to express temporal information. While several existing embedding-based methods [9, 18, 23] attempt to incorporate temporal point processes such as neural Hawkes process into their models, being capable of modeling the graph-wide link formation process in a temporal graph, such as Know-Evolve [22] models the occurrence of a fact as a multi-dimensional temporal point process for calculation in a multi-relation space. Mei *et al.* [18] present extensions to the multivariate Hawkes process using continuous-time LSTM to model the asynchronous event sequence. DyRep [23] presents an inductive framework based on the temporal point process, it addresses a different problem setting of a two-time scale with both association and communication events. As an event consequence, these methods cannot fully capture the continuous event evolving dynamics and prioritize different parts of the consequence, especially for complicated long-term event dependencies in the relevant historical event context, which limits the model performance.

Meanwhile, there has been a growing number of works on graph neural networks (GNNs) to explore the intrinsic topology relevance between entities or between entities and relations in TKG, so as to obtain high-quality embeddings, such as RE-NET [11] uses a multi-relational graph aggregator to capture the graph structural information. RE-GCN [15] employs RGCN to aggregate messages from neighboring entities and utilizes an auto-regressive GRU to model the temporal dependency among events. CRNet [26] leverages concurrent events from both historical and the future for TKGR. TGAP [12] explores query-relevant substructure in the TKG for path-based inference. These methods achieved good performance to known events in historical, recurrence, or periodicity dependency of relevant historical events to forecast events happening in the future. However, some events may have insufficient historical information and even be affected by other events during the whole timeline. As a result, it is a challenge to effectively model the hidden impact between these types of events and other events on TKG embedding learning. The temporal neighborhood event context naturally offers a viable solution for enhancing the comprehension of event occurrences in temporal graph learning models addressing the aforementioned problem.

Based on the aforementioned analysis, we should make use of both relevant historical events and temporal neighborhood event contexts. To obtain information from context adequately, in this paper, we proposed a new temporal knowledge graph reasoning model via jointly modeling relevant historical event and temporal neighborhood event context, (short-handed as TKGR-RHETNE). It contains an encoder and an aggregator to capture both the event evolution process and latent factors influencing the occurrence of an event in the relevant historical and temporal neighborhood context of events in the TKG, respectively. Finally, the model uses the above two parts of information to calcu-

late the conditional intensity and obtains the candidate entity score or time probability density of the next event.

In summary, our contributions are 3-fold as follows.

- We propose a new temporal knowledge graph reasoning model, called TKGR-RHETNE, which jointly models the relevant historical event and temporal neighborhood event context of events in the temporal knowledge graph.
- In the framework of the TKGR-RHETNE, we propose an encoder built on the transformer Hawkes process and self-attention mechanism to capture complex long-term event dependencies in relevant historical event context, thereby enabling continuous modeling of event evolution. We also present an aggregator that integrates the random walk strategy for modeling the potential influence of events with insufficient historical formation or are influenced by other events within their temporal neighborhood event context. The random walk strategy encompasses both the breadth-first random walk search, which aims to capture homophily information, and a depth-first search, designed to collect heterophily information.
- We carry out extensive experiments on five benchmark datasets to demonstrate the effectiveness of the TKGR-RHTNE in entity and timestamp forecasting tasks.

2 Related Work

2.1 Neural Hawkes Process-Based Method

Since the Hawkes process has been proven to model the exciting effects between events to capture the influence of historical events holistically, it is well suited for modeling the graph-wide link formation process in a temporal graph [22]. Neural Hawkes process [20] uses neural networks to parameterize the intensity function and exhibits high model capacity in complex real-life scenarios, e.g., Mei *et al.* [18] develop a neural Hawkes process based on continuous-time LSTM to model the asynchronous event sequence such that previous events are allowed to influence future events in complex and realistic ways. Know-Evolve [22] models the occurrence of a fact as a multi-dimensional temporal point process, it mainly learns non-linearly evolving entity representations by considering their interactions with other entities in multi-relational space, where the conditional intensity function of the process is adjusted based on the relationship score of the event. Zuo and Zhang *et al.* [28,30] apply the attention mechanism to the neural Hawkes process. However, these approaches fail to capture complicated dependencies well in the long-term event sequences.

2.2 Graph Neural Network-Based Method

Graph Neural Network (GNN)-based methods generally apply GNN to explore the intrinsic topology relevance between entities or between entities and relations in TKG. CyGNet [29], RE-NET [11], and RE-GCN [15] attempt to solve the entity prediction in the TKG from the perspective of each given query, which encodes the historical facts related to the subject entity in each query. These methods achieved good performance in entity prediction, but can not have the capacity of predicting timestamps in the

TKG. Some researchers employ temporal random walks to capture the continuous-time network dynamics, including CTDNE [19] based on time-respect random walks, and CAW-N [16] based on causal anonymous walks, however, they cannot collect homogeneous and heterogeneous information from the entire dynamic graph, which is implicitly subject to the homophily assumption.

3 The Proposed Model

The overall architecture of our proposed model is illustrated in Fig. 1. In the following, we provide a detailed introduction to three components and elaborate on the steps within each component.

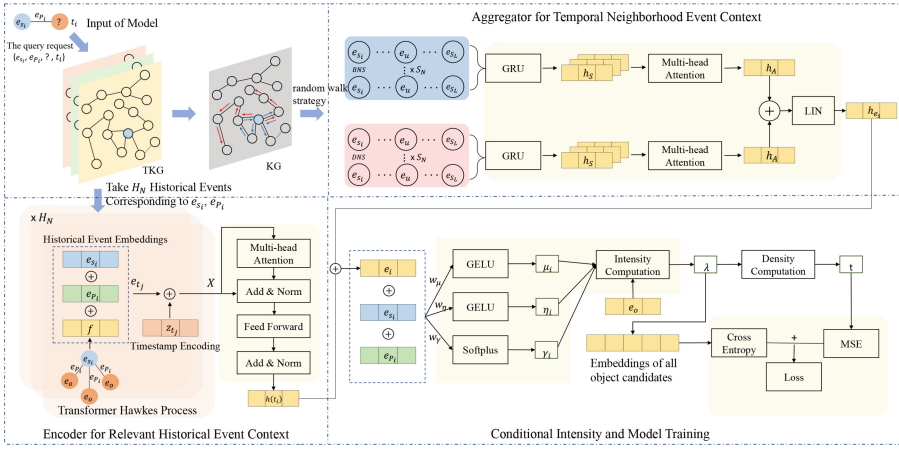


Fig. 1. The overall architecture of our proposed model.

3.1 Encoder for Relevant Historical Event Context

A TKG $\mathcal{G}$ can be formalized as a sequence of graph slices $\{\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_T\}$, where $\mathcal{G}_t = \{(e_s, e_p, e_o, t) \in \mathcal{G}\}$ denotes a graph slice that consists of facts that occurred at the timestamp t , i.e., events. Here, e_s and e_o represent the subject and object entities, respectively, and e_p denotes the predicate as a relation type.

Taking a query $(e_{s_i}, e_{p_i}, ?, t_i)$ in the TKG for an example, we assume an object forming relationship e_{p_i} with a given subject e_{s_i} at a timestamp t_i depends in part on past events involving e_{s_i} and e_{p_i} , this historical event encoder regards the past events and timestamps as relevant historical sequence $e_i^{h,sp}$ and time sequence $t_i^{h,sp}$:

$$e_i^{h,sp} = \left\{ \bigcup_{0 \leq t_j < t_i} \mathbf{O}_{t_j}(e_{s_i}, e_{p_i}) \right\}, t_i^{h,sp} = \left\{ \bigcup_{0 \leq t_j < t_i} t_j \right\}, \quad (1)$$

where $\mathbf{O}_{t_j}(e_{s_i}, e_{p_i})$ is a set of object entities that formed a link with the subject entity e_{s_i} under the predicate e_{p_i} at a timestamp t_j ($0 \leq t_j < t_i$), $\cup$ is the sign for the union. To preserve the occurrence order of the events, a.k.a timestamp, in the relevant historical time sequence $t_i^{h,sp}$, the event encoder adopts a temporal encoding procedure referring to the Transformer primitive location encoding method [25]. Specifically, the positions of the timestamps in the sequence are defined as:

$$[z_{t_j}]_i = \begin{cases} \cos\left(t_j/10000^{\frac{i-1}{d}}\right), & \text{if } i \text{ is odd,} \\ \sin\left(t_j/10000^{\frac{i}{d}}\right), & \text{if } i \text{ is even.} \end{cases} \quad (2)$$

For each $t_j \in t_i^{h,sp}$, we compute $\mathbf{z}_{t_j} \in \mathbb{R}^d$, where d is the dimension of encoding. Meanwhile, to learn the correlation between each event in the relevant historical event sequence $e_i^{h,sp}$, the historical event encoder represents each event as follows:

$$\mathbf{e}_{t_j} = \mathbf{W}_\lambda \left(\mathbf{e}_{s_i} \oplus \mathbf{f}\left(e_i^{h,sp}, t_j\right) \oplus \mathbf{e}_{p_i} \right), \quad (3)$$

where $\mathbf{f}\left(e_i^{h,sp}, t_i\right) \in \mathbb{R}^r$ denotes the average embedding of the object of the current subject at timestamp t_j , and $\oplus$ represents the concatenation operator. r denotes the rank of embeddings. Because the embeddings of the events and the encodings of timestamps are defined in spaces of different natures, to ensure alignment, the historical event encoder utilizes $\mathbf{W}_\lambda$ to map the event embeddings to the same space of the temporal encodings. Then, the encoder outputs the embeddings of the historical sequence $\mathcal{H} = \{(z_{t_j}, e_{t_j})\}$ to the subsequent self-attention module by computing $\mathbf{X} = (\mathbf{E} + \mathbf{Z})$.

The self-attention module enables the model to weigh and prioritize different parts of an input historical sequence $\mathcal{H}$, allowing it to capture complex long-range event dependencies and contextual information effectively. Specifically, we compute the attention output $\mathbf{A}$ by

$$\mathbf{A} = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{M_K}}\right)\mathbf{V}, \quad \text{where } \mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \mathbf{K} = \mathbf{X}\mathbf{W}^K, \mathbf{V} = \mathbf{X}\mathbf{W}^V. \quad (4)$$

Here $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ are the query, key, and value matrices obtained by different transformations of $\mathbf{X}$, and $\mathbf{W}^Q, \mathbf{W}^K \in \mathbb{R}^{d \times d_K}, \mathbf{W}^V \in \mathbb{R}^{d \times d_V}$ are the weights for the linear transformations, respectively. M_K is the variance of $\mathbf{Q}\mathbf{K}^\top$. The attention output $\mathbf{A}$ is then fed through a position-wise feed-forward neural network, generating hidden representations $\mathbf{h}(t)$ of the input historical event sequence:

$$\mathbf{H} = \text{ReLU}(\mathbf{A}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2, \quad \mathbf{h}(t_j) = \mathbf{H}(j, :), \quad (5)$$

where $\mathbf{W}_1 \in \mathbb{R}^{d \times d_H}, \mathbf{W}_2 \in \mathbb{R}^{d_H \times d}, \mathbf{b}_1 \in \mathbb{R}^{d_H}$, and $\mathbf{b}_2 \in \mathbb{R}^d$ are parameters of the neural network, and $\mathbf{W}_2$ has identical columns. The resulting matrix $\mathbf{H} \in \mathbb{R}^{L \times r}$ contains the hidden representation of all historical moments in the historical sequence, where each row corresponds to a specific historical moment. In practice, we employ multi-head self-attention to enhance the flexibility of the encoder, which also proves advantageous for data fitting. In addition, we incorporate masks into the attention mechanism to prevent it from accessing future information, i.e., when the attention produces an output for a specific position, we mask out all subsequent positions.

3.2 Aggregator for Temporal Neighborhood Event Context

Relying solely on the historical sequence of entities for the model reasoning can lead to diminished results, especially when confronted with insufficient historical information or disturbances from other events. The topological construction of TKG is influenced by the interactions between events. As such, integrating the graphical structure of TKG into event embeddings enables a more profound grasp of these underlying relationships.

We expect to capture the hidden semantic information associated with the temporal neighborhood events in the context of the neighborhood events of a given entity. Inspired by RAW-GNN [10], entity nodes in the neighborhood are divided according to whether they have similar behaviors with the given entity node. The hidden semantic information collected from these different entities can be divided into two categories, which we call homogeneous information and heterogeneous information.

In a slice of the temporal graph, we notice that examining the information of the broad neighborhood event sequence (BNS) of entities closely connected to or reachable with few jumps from a given entity, i.e., homogeneous information, which assists in understanding the potential impact of entity nodes with similar behaviors on a given entity, e.g., if an event is simultaneously initiated by several parallel events, extracting BNS information might be more likely to identify these latent relationships in breadth.

Besides, we can extract information from the deep neighborhood event sequence (DNS) of entities relatively distant from the designated entity, i.e., heterogeneous information. This aids in comprehending the potential impact of long or complex chains of relationships between a given entity and entities that behave differently. e.g., if an event is set off by a series of successive events, gathering DNS information might be more apt in capturing the underlying links within this extensive chain. The purpose of accumulating these two types of information is to thoroughly unearth the potential factors linked with the target event in the surrounding events of the specified target entity.

To illustrate this, we also shall consider an example consisting of a given query $(e_{s_i}, e_{p_i}, ?, t_i)$ in the TKGR, and perform the following operations:

Step 1. Obtain two types of event sequences. BNS and DNS originating from e_{s_i} can be obtained by applying breadth-first and depth-first search strategies on the graph. Consider a random walk that has traversed edge $p_{e_w e_u}$, is now on entity node e_u , and is going to visit the next entity node e_v . We set the unnormalized transition probability as follows:

$$S(e_{next} = e_v \mid e_{now} = e_u, e_{last} = e_w) = \begin{cases} 1/p, & \text{if } d_{e_w e_v} = 0 \\ 1, & \text{if } d_{e_w e_v} = 1 \\ 1/q, & \text{if } d_{e_w e_v} = 2 \\ 0, & \text{otherwise} \end{cases}, \quad (6)$$

where $d_{e_w e_v}$ is the length of the shortest path between nodes e_u and e_w . When we set $p < 1$ and $q > 1$, the random walk prefers to go back to the previous node, generating a BNS; and when $p > 1$ and $q < 1$, the random walk tends to generate a DNS.

Step 2. Aggregate event sequence information. Unlike the existing GNN-based methods that treat all neighboring entity nodes as an unordered set, the event sequence retains the ordered connections between nodes. Therefore, when aggregating information across all entity nodes in the sequence, we also take into account the order of the

entity nodes to preserve more relationship information between connected entities. The aggregation operation can be formulated as:

$$\mathbf{h}_S = GRU_{\theta}(\{\mathbf{h}_{e_1}, \mathbf{h}_{e_2}, \dots, \mathbf{h}_{e_K}\}), \quad (7)$$

where $\mathbf{h}_S \in \mathbb{R}^d$ is the embedding vector of sequence S , $\mathbf{h}_{e_k}^{(l)} \in \mathbb{R}^d$ is the feature of node e_k and θ represents all the learnable parameters in our message function GRU [3].

Step 3. Combine event sequence embeddings. Different sequences may have varying impacts on the target entity, therefore, we utilize an attention mechanism to determine the significance of each sequence embedding based on its relevance to the target entity.

$$\mathbf{h}_A = MultiHeadAttention_{\theta}(\mathbf{h}_S). \quad (8)$$

We use concatenation instead of sum to merge the sequential information embeddings from different strategies and feed them into a linear layer to obtain $\mathbf{h}_{e_s} \in \mathbb{R}^d$. Concatenation helps preserve unique information for each strategy.

Step 4. Obtain the final embeddings. Concatenating the representation of the historical information of the previous subsection with the representation of the neighborhood yields the final embedding used for the intensity function:

$$\mathbf{e}_i = (\mathbf{h}(t_i) \oplus \mathbf{h}_{e_s}). \quad (9)$$

3.3 Conditional Intensity

After generating final embedding of $\mathbf{e}_i$ at time t_i , we compute three parameters of the intensity function based on the $\mathbf{e}_{s_i}$, $\mathbf{e}_{p_i}$ and historical and neighbor hidden embedding $\mathbf{e}_i$:

$$\begin{aligned} \mu_i &= \text{GELU}((\mathbf{e}_i \oplus \mathbf{e}_{s_i} \oplus \mathbf{e}_{p_i}) W_{\mu}) \\ \eta_i &= \text{GELU}((\mathbf{e}_i \oplus \mathbf{e}_{s_i} \oplus \mathbf{e}_{p_i}) W_{\eta}), \\ \gamma_i &= \text{Softplus}((\mathbf{e}_i \oplus \mathbf{e}_{s_i} \oplus \mathbf{e}_{p_i}) W_{\gamma}) \end{aligned} \quad (10)$$

where η_i represents the initial intensity at time t_i , and μ_i represents the intensity when time tends to infinity. $(\eta_i - \mu_i)$ represents the rate at which the intensity changes, which can be both positive and negative. The function GELU represents the Gaussian Error Linear Unit for nonlinear activations. The Softplus is used for the decaying parameter since γ needs to be constrained to strictly positive values.

Finally, we express the intensity function of an object e_{o_i} forming relationship e_{p_i} with a given subject e_{s_i} at a timestamp t_i , as follows:

$$\lambda(e_{s_i}, e_{p_i}, t_i) = \text{Softplus}((\mu_i + (\eta_i - \mu_i) \exp(-\gamma_i(t - t_i))) \cdot \mathbf{e}_o), \quad (11)$$

for $t \in (t_i, t_{i+1}]$, where the Softplus is employed to constrain the intensity function to be positive.

3.4 Model Training and Prediction

Entity Prediction Task. Because all object candidates share the same survival term $\lambda(e_{s_i}, e_{p_i}, t)$ and the same value of t_L , we can directly compare their intensity function $\lambda(e_{s_i}, e_{p_i}, t_i)$. We use the cross-entropy loss for learning the entity prediction:

$$\mathcal{L}_{\text{entity}} = - \sum_{i=1}^N \sum_{c=1}^{N_e} y_c \log(p(e_{o_i} = c \mid e_{s_i}, e_{p_i}, t_i)). \quad (12)$$

Time Prediction Task. We use the mean square error as the timestamp prediction loss:

$$\mathcal{L}_{\text{time}} = \sum_{i=1}^N (t_i - \hat{t}_i)^2. \quad (13)$$

Joint Learning. Different types of loss functions are linearly combined to jointly learn two tasks in an end-to-end manner:

$$\mathcal{L} = \lambda_e \mathcal{L}_e + \lambda_t \mathcal{L}_t + \lambda_l \|\Theta\|_2^2, \quad (14)$$

where λ_e , λ_t , and λ_l are weights that balance the importance of two losses. Θ contains all model parameters.

4 Experiments and Analysis

4.1 Experimental Settings and Implementation Details

Datasets. We evaluate our model and baselines on five datasets, including ICEWS14 [6], ICEWS18 [11], ICEWS05-15 [6], YAGO [17] and WIKI [14]. To ensure a fair comparison, we use the split manner provided by Sun *et al.* [21].

Baselines. For the entity prediction task, we compare our model with three categories of KG embedding models: (1) *static KG embedding models*, including TransE [2]¹, DistMult [27]¹, and ComplEx [24]¹. We apply these models in static KGs that ignore timestamp information. (2) *TKGR models*, including TTransE [14]², TA-DistMult [6], DE-Simple [7], TNTComplEx [13], RE-NET [11], CyGNet [29], RE-GCN [15] and TANGO [8]. (3) *TKGR models with Hawkes process* are most related to our model, including GHNN [9] and GHT [21]. For the timestamp prediction task, we compare our model with one temporal point process model for TKGR, i.e., GHNN [9].

Evaluation Metrics. The mean reciprocal rank (MRR) and Hits@k are standard metrics for the entity prediction task. MRR is the average reciprocal of the correct query answer rank. Hits@k indicates the proportion of correct answers among the top k candidates. For the timestamp prediction task, we use the mean absolute error (MAE) between the predicted timestamp and ground truth timestamp as the metric.

¹ The code for TransE, DistMult, and ComplEx is from <https://github.com/thunlp/OpenKE>.

² The code for TTransE is from <https://github.com/INK-USC/RE-Net/tree/master/baselines>.

Details of Baselines. We directly adopt the results of static KG embedding models, interpolated reasoning models, and CyGNet reported in earlier works [8]. For TANGO³, RE-NET⁴, RE-GCN⁵, GHNN⁶, and GHT⁷, we use their released source codes with default hyperparameters. We rerun these models on four benchmark datasets. To ensure fairness, for RE-GCN, we don't use the module that encodes the node type information.

Details of TKGR-RHETNE. We set the dimension of all embeddings and hidden states to 200. The relevant historical sequence length for the encoder is limited to 10, and the attention head number is set to 3. For the aggregator for temporal neighborhood event context, the number and length of sequences for each strategy are 8 and 6, respectively, the GRU layer number is 1, and the attention head number is 2. In total, we train our model for 200 epochs and use the Adam optimizer with parameters, with a learning rate of 0.003 and a weight decay of 1e-5. The batch size is set to 256.

4.2 Experimental Results and Discussion

Entity Prediction. Table 1 and Table 2 report the experimental results of the entity prediction task on five TKGR datasets. We highlight the best results in bold. On the three ICEWS datasets, static KG embedding methods fell far behind our model due to their inability to capture temporal dynamics. Our model is superior to the heuristic evolutionary methods of RE-NET and RE-GCN in predicting events because their learning effect in the later time is highly dependent on the learning effect in the earlier time and will forget the previous knowledge in the process of evolution. By contrast, we introduce an encoder based on the transformer Hawkes process when encoding relevant historical event context, and it can learn the influence of the current event from the event occurring at any distance from the current time in parallel, which helps the model capture complex long-term event dependencies for modeling the event evolution continuously. Unlike existing GNN-based TKGR models, which often imply homogeneity assumptions, our model considers different aggregation strategies to collect latent factors influencing the occurrence of an event in temporal neighborhood events, which is the reason why it performs better than them. On the YAGO and WIKI datasets, it can be observed that there is a significant improvement in the performance of our model, with a boost of 13.85% and 7.47% on Hits@1. The reason for this phenomenon is that for WIKI and YAGO, a large number of quadruples contain unseen entities in the test set. This prevents many models from learning from historical information when inferencing entities, but the encoder for temporal neighborhood event context with different aggregation strategies in our model can effectively alleviate this problem, and the probability of selecting irrelevant entities is greatly reduced.

³ The code for TANGO is from <https://github.com/TemporalKGTeam/TANGO>.

⁴ The code for RE-NET is from <https://github.com/INK-USC/RE-Net>.

⁵ The code for RE-GCN is from <https://github.com/Lee-zix/RE-GCN>.

⁶ The code for GHNN is from https://github.com/Jeff20100601/GHNN_clean.

⁷ The code for GHT is from <https://github.com/JHL-HUST/GHT>.

Table 1. Experimental results of entity prediction on ICEWS-style dataset. The best result in each column is boldfaced.

Models	ICEWS14				ICEWS18				ICEWS05-15			
	MRR	Hits@1	Hits@3	Hits@10	MRR	Hits@1	Hits@3	Hits@10	MRR	Hits@1	Hits@3	Hits@10
TransE	22.48	13.36	25.63	41.23	12.24	5.84	12.81	25.10	22.55	13.05	25.61	42.05
Distmult	27.67	18.16	31.15	46.96	10.17	4.52	10.33	21.25	28.73	19.33	32.19	47.54
ComplEx	30.84	21.51	34.48	49.59	21.01	11.87	23.47	39.97	31.69	21.44	35.74	52.04
TTransE	13.43	3.11	17.32	34.55	8.31	1.92	8.56	21.89	15.71	5.00	19.72	38.02
TA-DistMult	26.47	17.09	30.22	45.41	16.75	8.61	18.41	33.59	24.31	14.58	27.92	44.21
DE-Simple	32.67	24.43	35.69	49.11	19.30	11.53	21.86	34.80	35.02	25.91	38.99	52.75
TNTComplEx	32.12	23.35	36.03	49.13	21.23	13.28	24.02	36.91	27.54	19.52	30.80	42.86
CyGNet	32.73	23.69	36.31	50.67	24.93	15.90	28.28	42.61	34.97	25.67	39.09	52.94
TANGO-TuckER	26.25	17.30	29.07	44.18	28.97	19.51	32.61	47.51	42.86	32.73	48.14	62.34
TANGO-Distmult	24.70	16.36	27.26	41.35	27.56	18.68	30.86	44.94	40.71	31.23	45.33	58.95
RE-NET	35.71	26.79	39.57	52.81	26.88	17.70	30.35	45.04	39.11	29.04	44.10	58.90
RE-GCN	36.08	27.30	39.89	53.01	27.34	18.44	30.48	44.85	39.89	30.10	44.63	56.98
GHNN	28.68	19.74	31.64	46.47	28.12	19.05	31.62	45.72	41.41	32.48	45.63	58.55
GHT	37.40	27.77	41.66	56.19	27.40	18.08	30.76	45.76	41.50	30.79	46.85	62.73
TKGR-RHETNE	37.50	28.23	41.95	55.09	29.26	19.80	33.03	47.66	43.30	32.48	48.08	64.83

Table 2. Experimental results of entity prediction on YAGO and WIKI datasets. The best result in each column is boldfaced.

Models	YAGO				WIKI			
	MRR	Hits@1	Hits@3	Hits@10	MRR	Hits@1	Hits@3	Hits@10
TransE	48.97	36.19	62.45	66.05	46.68	46.23	49.71	51.71
Distmult	54.84	47.39	59.81	68.52	49.66	46.17	52.81	54.13
ComplEx	61.29	54.88	62.28	66.82	47.84	38.15	50.08	51.39
TTransE	31.19	18.12	40.91	51.21	29.27	21.67	34.43	42.39
TA-DistMult	54.92	48.15	59.61	66.71	44.53	39.92	48.73	51.71
DE-Simple	54.91	51.64	57.30	60.17	45.43	42.60	47.71	49.55
TNTComplEx	57.98	52.92	61.33	66.69	45.03	40.04	49.31	52.03
CyGNet	52.07	45.36	56.12	63.77	33.89	29.01	36.10	41.86
TANGO-TuckER	62.50	58.77	64.73	68.63	51.60	49.61	52.45	54.87
TANGO-Distmult	63.34	60.04	65.19	68.79	53.04	51.52	53.84	55.46
RE-NET	58.02	53.06	61.08	66.29	49.66	46.88	51.19	53.48
RE-GCN	58.27	59.98	65.62	75.94	39.84	39.82	44.43	53.88
GHNN	62.79	60.29	63.93	66.85	-	-	-	-
GHT	64.60	57.67	69.39	76.95	58.85	55.22	61.53	64.72
TKGR-RHETNE	79.19	74.14	83.71	85.76	66.17	62.69	68.03	72.22

Timestamp Prediction. In the timestamp prediction task, our model exhibits mean absolute errors of 1.76, 14.88, and 4.88 on the ICEWS14, ICEWS18, and ICEWS05-15 datasets, respectively. These results surpass those of GHNN by margins of 4.33, 6.67, and 2.86, underscoring the enhanced capabilities of our model, which leverages the transformer for constructing the Hawkes process. Conversely, GHNN employs a continuous-time LSTM to gauge the intensity function. However, this approach neglects to directly integrate timestamp information and falls short in capturing intricate long-term relationships. Consequently, our model consistently outshines GHNN in the timestamp prediction task.

4.3 Ablation Study

In this subsection, we choose YAGO to investigate the effectiveness of different aggregation strategies for entity prediction in the aggregator for temporal neighborhood event context. Specifically, we separately experiment with the impact of using only the deep aggregation strategy, and the breadth aggregation strategy, and not using the aggregator for temporal neighborhood event context on the model performance. Table 3 shows the ablation results. As can be seen from the results in the table, using only the depth strategy to aggregate neighbor information results in a decrease of 3.07% on Hits@1. This is because the model lacks the capture of concurrent events in the temporal neighborhood event context or the potential impact of entities with similar behavior on the current entity. If only the breadth strategy is used to aggregate neighbor information, the model loses its ability to capture complex information for longer event sequences, resulting in a decrease of 1.68% in the results on Hits@1. Therefore, it is necessary to use both aggregation strategies to ensure the best model performance.

Table 3. Ablation study of our model on YAGO dataset.

Models	YAGO			
	MRR	Hits@1	Hits@3	Hits@10
TKGR-RHTNE-w/o - DNS & BNS	76.44	70.75	81.20	85.03
TKGR-RHTNE-w/o - DNS	77.74	72.46	82.48	84.91
TKGR-RHTNE-w/o - BNS	76.28	71.07	81.11	84.79
TKGR-RHTNE	79.19	74.14	83.71	85.76

5 Conclusion

In this paper, we introduce a novel TKGR model, named TKGR-RHETNE, considering the modeling in the context of relevant historical events and temporal neighborhood events. Utilizing an encoder built on the transformer Hawkes process and self-attention mechanism, this model can effectively grasp complex long-term event dependencies in the relevant historical event context, enabling continuous modeling of event evolution.

Additionally, employing a neighborhood aggregator designed to integrate the random walk strategy within the TKG topological structure, the model can learn the potential influence between events with insufficient historical information and other events for modeling the temporal neighborhood event context. Our experimental results on five popular datasets demonstrate the superior performance of our model on both entity prediction and timestamp prediction tasks. Future work includes incorporating logical rules into the proposed model to enhance its interpretability in the TKGR procedure.

Acknowledgements. This work was supported by the National Natural Science Foundation of China (Grant No. 62202075, No. 62171111, No. 62376058, No. 62376043, No. 62002052), the Natural Science Foundation of Chongqing, China (2022NSCQ-MSX3749), Sichuan Science and Technology Program (Grant No. 2022YFG0189), China Postdoctoral Science Foundation (Grant No. 2022M710614), Key Laboratory of Data Science and Smart Education, Hainan Normal University, Ministry of Education (Grant No. 2022NSCQ-MSX3749), Anhui Provincial Engineering Laboratory for Beidou Precision Agriculture Information (Grant No. BDSY2023004).

References

1. Barbosa, D., Wang, H., Yu, C.: Shallow information extraction for the knowledge web. In: ICDE, pp. 1264–1267 (2013)
2. Bordes, A., Usunier, N., Garcia-Duran, A., Weston, J., Yakhnenko, O.: Translating embeddings for modeling multi-relational data. In: NIPS, pp. 2787–2795 (2013)
3. Cho, K., et al.: Learning phrase representations using rnn encoder-decoder for statistical machine translation. arXiv preprint [arXiv:1406.1078](https://arxiv.org/abs/1406.1078) (2014)
4. Deng, S., Rangwala, H., Ning, Y.: Dynamic knowledge graph based multi-event forecasting. In: SIGKDD, pp. 1585–1595 (2020)
5. Feng, F., He, X., Wang, X., Luo, C., Liu, Y., Chua, T.S.: Temporal relational ranking for stock prediction. ACM Trans. Inform. Syst. (TOIS) **37**(2), 1–30 (2019)
6. García-Durán, A., Dumančić, S., Niepert, M.: Learning sequence encoders for temporal knowledge graph completion. arXiv preprint [arXiv:1809.03202](https://arxiv.org/abs/1809.03202) (2018)
7. Goel, R., Kazemi, S.M., Brubaker, M., Poupart, P.: Diachronic embedding for temporal knowledge graph completion. In: AAAI, vol. 34, pp. 3988–3995 (2020)
8. Han, Z., Ding, Z., Ma, Y., Gu, Y., Tresp, V.: learning neural ordinary equations for forecasting future links on temporal knowledge graphs. In: EMNLP, pp. 8352–8364 (2021)
9. Han, Z., Ma, Y., Wang, Y., Günnemann, S., Tresp, V.: Graph hawkes neural network for forecasting on temporal knowledge graphs. arXiv preprint [arXiv:2003.13432](https://arxiv.org/abs/2003.13432) (2020)
10. Jin, D., et al.: Raw-gnn: Random walk aggregation based graph neural network. arXiv preprint [arXiv:2206.13953](https://arxiv.org/abs/2206.13953) (2022)
11. Jin, W., Qu, M., Jin, X., Ren, X.: Recurrent event network: Autoregressive structure inference over temporal knowledge graphs. arXiv preprint [arXiv:1904.05530](https://arxiv.org/abs/1904.05530) (2019)
12. Jung, J., Jung, J., Kang, U.: Learning to walk across time for interpretable temporal knowledge graph completion. In: SIGKDD, pp. 786–795 (2021)
13. Lacroix, T., Obozinski, G., Usunier, N.: Tensor decompositions for temporal knowledge base completion. arXiv preprint [arXiv:2004.04926](https://arxiv.org/abs/2004.04926) (2020)
14. Leblay, J., Chekol, M.W.: Deriving validity time in knowledge graph. In: The Web Conference, pp. 1771–1776 (2018)
15. Li, Z., et al.: Temporal knowledge graph reasoning based on evolutionary representation learning. In: SIGIR, pp. 408–417 (2021)

16. Liu, M., Liu, Y.: Inductive representation learning in temporal networks via mining neighborhood and community influences. In: SIGIR, pp. 2202–2206 (2021)
17. Mahdisoltani, F., Biega, J., Suchanek, F.: Yago3: a knowledge base from multilingual wikipedias. In: 7th Biennial Conference on Innovative Data Systems Research. CIDR Conference (2014)
18. Mei, H., Eisner, J.M.: The neural hawkes process: a neurally self-modulating multivariate point process. In: NeurIPS 30 (2017)
19. Nguyen, G.H., Lee, J.B., Rossi, R.A., Ahmed, N.K., Koh, E., Kim, S.: Continuous-time dynamic network embeddings. In: The Web Conference, pp. 969–976 (2018)
20. Shchur, O., Türkmen, A.C., Januschowski, T., Günnemann, S.: Neural temporal point processes: a review. arXiv preprint [arXiv:2104.03528](https://arxiv.org/abs/2104.03528) (2021)
21. Sun, H., Geng, S., Zhong, J., Hu, H., He, K.: Graph hawkes transformer for extrapolated reasoning on temporal knowledge graphs. In: EMNLP, pp. 7481–7493 (2022)
22. Trivedi, R., Dai, H., Wang, Y., Song, L.: Know-evolve: deep temporal reasoning for dynamic knowledge graphs. In: ICML, pp. 3462–3471. PMLR (2017)
23. Trivedi, R., Farajtabar, M., Biswal, P., Zha, H.: Dyrep: learning representations over dynamic graphs. In: ICLR (2019)
24. Trouillon, T., Welbl, J., Riedel, S., Gaussier, É., Bouchard, G.: Complex embeddings for simple link prediction. In: ICML, pp. 2071–2080 (2016)
25. Vaswani, A., et al.: Attention is all you need. In: NeurIPS, vol. 30 (2017)
26. Wang, S., Cai, X., Zhang, Y., Yuan, X.: CRNet: modeling concurrent events over temporal knowledge graph. In: Sattler, U., et al. (eds.) The Semantic Web – ISWC 2022: 21st International Semantic Web Conference, Virtual Event, October 23–27, 2022, Proceedings, pp. 516–533. Springer International Publishing, Cham (2022). https://doi.org/10.1007/978-3-031-19433-7_30
27. Yang, B., Yih, W.t., He, X., Gao, J., Deng, L.: Embedding entities and relations for learning and inference in knowledge bases. <https://arxiv.org/abs/1412.6575> (2014)
28. Zhang, Q., Lipani, A., Kirnap, O., Yilmaz, E.: Self-attentive hawkes process. In: ICML, pp. 11183–11193. PMLR (2020)
29. Zhu, C., Chen, M., Fan, C., Cheng, G., Zhang, Y.: Learning from history: modeling temporal knowledge graphs with sequential copy-generation networks. In: AAAI. vol. 35, pp. 4732–4740 (2021)
30. Zuo, S., Jiang, H., Li, Z., Zhao, T., Zha, H.: Transformer hawkes process. In: ICML, pp. 11692–11702. PMLR (2020)